

CO4 ASSESSMENT TOOL 1

CASE STUDY ASSESSMENT

NAME: N. PARNITA

REG.NO: 192525322

COURSE: Design and Analysis of Algorithms – CSA0617

Case Study 1: Stock Market Analysis System Using Quick Sort

1. Title

Stock Market Analysis System Using Quick Sort

2. Aim

To implement the **Quick Sort algorithm** using the **Divide and Conquer** approach for efficiently sorting stock prices and analyze its performance.

3. Content

Case Study

A stock market analysis system receives a large number of stock prices that need to be sorted for analysis. Quick Sort is an efficient sorting algorithm based on the **Divide and Conquer** technique.

The algorithm selects a **pivot element** and divides the array into two parts. Elements smaller than the pivot are placed on one side, while larger elements are placed on the other side. The same process is recursively applied to the two partitions.

Working

1. Store the stock prices in an array.
2. Select the last element as the **pivot**.
3. Partition the array around the pivot.
4. Place smaller values before the pivot and larger values after it.
5. Recursively apply Quick Sort to both partitions.
6. Display the sorted stock prices.

Complexity

- **Best Case:** $O(n \log n)$
- **Average Case:** $O(n \log n)$
- **Worst Case:** $O(n^2)$
- **Space Complexity:** $O(\log n)$ on average.

Quick Sort provides efficient performance for sorting large amounts of stock price data, especially when a suitable pivot is selected.

4.Python Code

```
import time
```

```
# Quick Sort using Divide and Conquer
```

```
def quick_sort(arr):
```

```
    # Base case
```

```
    if len(arr) <= 1:
```

```
        return arr
```

```
    # Select the last element as pivot
```

```
    pivot = arr[-1]
```

```
# Divide the array
```

```
left = [x for x in arr[:-1] if x <= pivot]
```

```
right = [x for x in arr[:-1] if x > pivot]
```

```
# Conquer and combine
```

```
return quick_sort(left) + [pivot] + quick_sort(right)
```

```
# Stock prices
```

```
stock_prices = [245, 180, 320, 150, 275, 210, 400, 195]
```

```
print("Original Stock Prices:")
```

```
print(stock_prices)
```

```
# Measure execution time
```

```
start = time.perf_counter()
```

```
sorted_prices = quick_sort(stock_prices)
```

```
end = time.perf_counter()
```

```
execution_time = end - start
```

```
print("\nSorted Stock Prices:")  
  
print(sorted_prices)  
  
print("\nExecution Time:")  
  
print(execution_time, "seconds")
```

5. Sample Output

The execution time may vary slightly depending on your computer.

Original Stock Prices:

[245, 180, 320, 150, 275, 210, 400, 195]

Sorted Stock Prices:

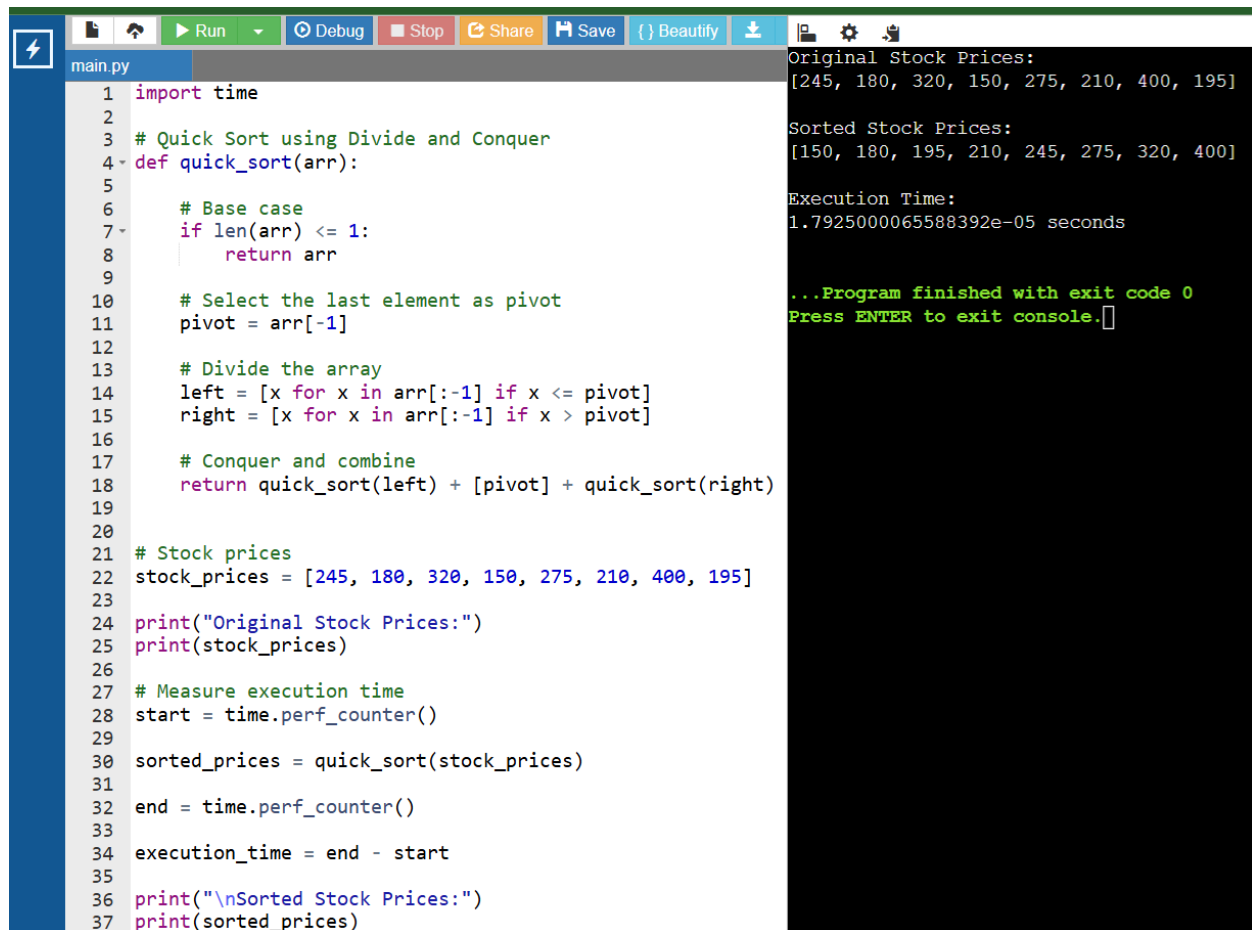
[150, 180, 195, 210, 245, 275, 320, 400]

Execution Time:

0.0000... seconds

6. Result

The stock prices were successfully sorted using the **Quick Sort algorithm**. The experiment demonstrates the **Divide and Conquer approach**, where the stock price data is divided around a pivot and recursively sorted. Quick Sort provides **$O(n \log n)$ average-case performance**, making it suitable for efficient stock market data analysis.



The screenshot shows a Python IDE with a file named `main.py`. The code implements a Quick Sort algorithm. It starts with an import of the `time` module. A `quick_sort` function is defined, which uses a divide-and-conquer approach. The base case is when the array length is 1 or less, in which case it returns the array. Otherwise, it selects the last element as the pivot, partitions the array into elements less than or equal to the pivot and elements greater than the pivot, and then recursively sorts both sub-arrays. The main part of the code defines a list of stock prices, prints them, measures the execution time using `time.perf_counter()`, sorts the prices using the `quick_sort` function, and prints the sorted prices. The output on the right shows the original and sorted stock prices, the execution time in scientific notation, and a message indicating the program finished successfully.

```
1 import time
2
3 # Quick Sort using Divide and Conquer
4 def quick_sort(arr):
5
6     # Base case
7     if len(arr) <= 1:
8         return arr
9
10    # Select the last element as pivot
11    pivot = arr[-1]
12
13    # Divide the array
14    left = [x for x in arr[:-1] if x <= pivot]
15    right = [x for x in arr[:-1] if x > pivot]
16
17    # Conquer and combine
18    return quick_sort(left) + [pivot] + quick_sort(right)
19
20
21 # Stock prices
22 stock_prices = [245, 180, 320, 150, 275, 210, 400, 195]
23
24 print("Original Stock Prices:")
25 print(stock_prices)
26
27 # Measure execution time
28 start = time.perf_counter()
29
30 sorted_prices = quick_sort(stock_prices)
31
32 end = time.perf_counter()
33
34 execution_time = end - start
35
36 print("\nSorted Stock Prices:")
37 print(sorted_prices)
```

Original Stock Prices:
[245, 180, 320, 150, 275, 210, 400, 195]

Sorted Stock Prices:
[150, 180, 195, 210, 245, 275, 320, 400]

Execution Time:
1.7925000065588392e-05 seconds

...Program finished with exit code 0
Press ENTER to exit console.

Case Study 2: Image Compression Engine Using Matrix Multiplication

1. Title

Image Compression Engine Using Matrix Multiplication

2. Aim

To implement **Matrix Multiplication using Strassen's Divide and Conquer algorithm** and analyze its efficiency for image transformation and compression applications.

3. Content

Case Study

An image can be represented as a matrix containing pixel values. Matrix multiplication is used in image processing for transformations and compression operations. For large matrices, conventional matrix multiplication requires $O(n^3)$ time.

Strassen's algorithm uses the **Divide and Conquer** approach. It divides the matrices into smaller submatrices and reduces the number of multiplications from 8 to 7 at each recursive step.

Working

1. Represent the image data as a matrix.
2. Divide the matrices into smaller submatrices.
3. Perform seven multiplication operations instead of eight.
4. Combine the resulting submatrices.
5. Obtain the final transformed matrix.
6. Compare the result with conventional matrix multiplication.

Complexity

- **Traditional Matrix Multiplication:** $O(n^3)$
- **Strassen's Algorithm:** $O(n^{\log_2 7}) \approx O(n^{2.81})$
- Strassen's algorithm is more efficient for **large matrices**.

4. Python Code

```
import numpy as np
```

```
import time
```

```
# Strassen Matrix Multiplication
```

```
def strassen(A, B):

    n = len(A)

    # Base case
    if n == 1:
        return A * B

    # Divide matrices into 4 parts
    mid = n // 2

    A11 = A[:mid, :mid]
    A12 = A[:mid, mid:]
    A21 = A[mid:, :mid]
    A22 = A[mid:, mid:]

    B11 = B[:mid, :mid]
    B12 = B[:mid, mid:]
    B21 = B[mid:, :mid]
    B22 = B[mid:, mid:]

    # Seven Strassen multiplications
    M1 = strassen(A11 + A22, B11 + B22)
```

```
M2 = strassen(A21 + A22, B11)
```

```
M3 = strassen(A11, B12 - B22)
```

```
M4 = strassen(A22, B21 - B11)
```

```
M5 = strassen(A11 + A12, B22)
```

```
M6 = strassen(A21 - A11, B11 + B12)
```

```
M7 = strassen(A12 - A22, B21 + B22)
```

```
# Calculate result submatrices
```

```
C11 = M1 + M4 - M5 + M7
```

```
C12 = M3 + M5
```

```
C21 = M2 + M4
```

```
C22 = M1 - M2 + M3 + M6
```

```
# Combine submatrices
```

```
C = np.vstack((  
    np.hstack((C11, C12)),  
    np.hstack((C21, C22))  
))
```

```
return C
```

```
# Image matrix
```



```
A = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12],
    [13, 14, 15, 16]
])
```

```
# Transformation matrix
```

```
B = np.array([
    [1, 0, 0, 1],
    [0, 1, 1, 0],
    [1, 0, 1, 0],
    [0, 1, 0, 1]
])
```

```
# Normal Matrix Multiplication
```

```
start = time.perf_counter()
```

```
normal_result = A @ B
```

```
normal_time = time.perf_counter() - start
```

```
# Strassen Matrix Multiplication
```

```
start = time.perf_counter()
```

```
strassen_result = strassen(A, B)
```

```
strassen_time = time.perf_counter() - start

# Display results
print("Image Matrix A:")
print(A)

print("\nTransformation Matrix B:")
print(B)

print("\nNormal Matrix Multiplication Result:")
print(normal_result)

print("\nStrassen Matrix Multiplication Result:")
print(strassen_result)

print("\nResults are equal:",
      np.array_equal(normal_result, strassen_result))

print("\nNormal Multiplication Time:",
      normal_time, "seconds")

print("Strassen Multiplication Time:",
      strassen_time, "seconds")
```

5. Sample Output

Your exact execution time will be different depending on your computer, but the matrix results should look like this:

Image Matrix A:

```
[[ 1 2 3 4]
```

```
[ 5 6 7 8]
```

```
[ 9 10 11 12]
```

```
[13 14 15 16]]
```

Transformation Matrix B:

```
[[1 0 0 1]
```

```
[0 1 1 0]
```

```
[1 0 1 0]
```

```
[0 1 0 1]]
```

Normal Matrix Multiplication Result:

```
[[ 4 7 5 5]
```

```
[12 15 13 13]
```

```
[16 19 17 21]
```

```
[28 31 29 29]]
```

Strassen Matrix Multiplication Result:

```
[[ 4 7 5 5]
```

[12 15 13 13]

[16 19 17 21]

[28 31 29 29]]

Results are equal: True

Normal Multiplication Time: 0.0000... seconds

Strassen Multiplication Time: 0.000... seconds

6. Result

The image matrix was successfully multiplied using **Strassen's Matrix Multiplication algorithm**. The results were verified against conventional matrix multiplication and found to be identical. The experiment demonstrates the use of the **Divide and Conquer technique** and shows how Strassen's algorithm can reduce the computational complexity of matrix multiplication for large matrices.

```
main.py
1 import numpy as np
2 import time
3
4 # Strassen Matrix Multiplication
5 def strassen(A, B):
6
7     n = len(A)
8
9     # Base case
10    if n == 1:
11        return A * B
12
13    # Divide matrices into 4 parts
14    mid = n // 2
15
16    A11 = A[:mid, :mid]
17    A12 = A[:mid, mid:]
18    A21 = A[mid:, :mid]
19    A22 = A[mid:, mid:]
20
21    B11 = B[:mid, :mid]
22    B12 = B[:mid, mid:]
23    B21 = B[mid:, :mid]
24    B22 = B[mid:, mid:]
25
26    # Seven Strassen multiplications
27    M1 = strassen(A11 + A22, B11 + B22)
28    M2 = strassen(A21 + A22, B11)
29    M3 = strassen(A11, B12 - B22)
30    M4 = strassen(A22, B21 - B11)
31    M5 = strassen(A11 + A12, B22)
32    M6 = strassen(A21 - A11, B11 + B12)
33    M7 = strassen(A12 - A22, B21 + B22)
34
35    # Calculate result submatrices
36    C11 = M1 + M4 - M5 + M7
37    C12 = M3 + M5
```

```
Image Matrix A:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]]

Transformation Matrix B:
[[1 0 0 1]
 [0 1 1 0]
 [1 0 1 0]
 [0 1 0 1]]

Normal Matrix Multiplication Result:
[[ 4  6  5  5]
 [12 14 13 13]
 [20 22 21 21]
 [28 30 29 29]]

Strassen Matrix Multiplication Result:
[[ 4  6  5  5]
 [12 14 13 13]
 [20 22 21 21]
 [28 30 29 29]]

Results are equal: True

Normal Multiplication Time: 9.450699963053921e-05 seconds
Strassen Multiplication Time: 0.0006624929992540274 seconds

...Program finished with exit code 0
Press ENTER to exit console.
```